Releasing

Super Submitter for macOS. Everything here happens in an app or in a web page.

Sending the code to GitHub builds a signed, notarized release and publishes it. The app then finds that release on its own, because Sparkle polls the appcast the release carries.

Nothing ships without you. Every run stops and waits for your approval. You press a button on the Actions page, and only then does it build.

Already set up. You do not need to do any of this.

- The repository **rafacst/app-super-submitter** exists and is public.
- Your Mac already knows how to reach it. The remote is named `github`.
- The approval gate exists. The environment is named `release` and you are the required reviewer.
- The Sparkle signing key already exists in your login keychain, and the app already carries its matching public half. **Never create a new one.**
- Your Mac is already signed in to GitHub, so sending code will not ask for a password.

What is left is five secrets, one push, and one approval. Steps 1 to 5 happen once. Steps 6 and 7 are how you ship, every time.

The repository name is permanent. Its address is inside every copy of the app you ever release, and a released copy asks that address for updates forever. If you rename or delete the repository, every installed copy stops updating and there is no way to tell it where to look. This already happened once, to the previous repository. Leave the name alone.

Set it up once

1 Copy the Sparkle signing key Keychain Access

This key proves an update came from you. The app refuses any download this key did not sign.

1. Press **Command** and **Space**, type **Keychain Access**, and press Return.
2. In the sidebar choose the **login** keychain, then the **All Items** category.
3. Type **sparkle** in the search box at the top right.
4. Double-click the item named **https://sparkle-project.org**. Its Kind is *private key* and its Account is *ed25519*.
5. Tick **Show password**. Your Mac asks for your login password. Type it and choose **Allow**.
6. Select the line of text that appears and copy it. It is 44 characters and it ends in `=`.

Paste it somewhere safe for a moment. You need it in step 5, as `SPARKLE_PRIVATE_KEY`.

> **This key cannot be replaced.** If you ever lose it, no copy of the app that people already have can update again, ever. Back up that keychain item. Never run Sparkle's key generator a second time: a new key makes every installed copy refuse every update you publish, silently.

2 Export the signing certificate Keychain Access

This is the Apple certificate that tells macOS the app is really yours.

1. Stay in Keychain Access. Clear the search box.
2. In the sidebar choose the **login** keychain, then the **My Certificates** category. This category matters. The plain **Certificates** category leaves out the private half, and the export will not work.

3. Find **Developer ID Application: R CASTRO SILVA CONSULTORIA EM TECNOLOGIA LTDA (88BXH8KNVZ)**.
4. Right-click it and choose **Export**.
5. Set the file format to **Personal Information Exchange (.p12)**. Save it to your Desktop as **certificate.p12**.
6. Your Mac asks you to invent a password for the file. Type one and write it down. You need it in step 5, as `DEVELOPER_ID_CERT_PASSWORD`.
7. Your Mac then asks for your login password, to allow the export. Type it and choose **Allow**.

3 Turn the certificate into text Shortcuts

A GitHub secret holds text, and `certificate.p12` is not text. This step converts it. It is the only step on this page with no app built for the job, so you build a small one yourself, once. It takes about a minute and you can use it again forever.

1. Open the **Shortcuts** app and choose **File**, then **New Shortcut**.
2. In the search box on the right, type **base64**. Drag the **Base64 Encode** action into the empty shortcut.
3. Search for **clipboard**. Drag **Copy to Clipboard** underneath it.
4. Open the shortcut's settings in the sidebar on the right and turn on **Use as Quick Action** and **Finder**. Name it **Copy as Base64** and close the window.
5. In Finder, right-click **certificate.p12**, open **Quick Actions**, and choose **Copy as Base64**.

The clipboard now holds a long block of letters and numbers. Paste it somewhere safe. You need it in step 5, as `DEVELOPER_ID_CERT_P12`.

IF YOU WOULD RATHER NOT BUILD THE SHORTCUT

Open the Terminal app and paste this one line. It does the same thing and puts the result on the clipboard.

```
base64 -i ~/Desktop/certificate.p12 | pbcopy
```

4 Make an app-specific password appleid.apple.com

Apple checks every release before people can open it. That check signs in as you, and it needs a password of its own. Your real Apple ID password will not work.

1. Go to appleid.apple.com and sign in.
2. Open **Sign-In and Security**, then **App-Specific Passwords**.
3. Choose the **+** button, name it **Super Submitter releases**, and confirm.
4. Copy the password Apple shows you. It looks like `abcd-efgh-ijkl-mnop`. Apple shows it once only.

You need it in step 5, as `NOTARY_PASSWORD`.

5 Add the five secrets github.com

1. Go to **github.com/rafacst/app-super-submitter**.
2. Open the **Settings** tab, then **Secrets and variables** in the sidebar, then **Actions**.
3. Choose **New repository secret**. Type the name, paste the value, and choose **Add secret**.
4. Do that five times, once for each row below. The names must match exactly.

Name	What to paste
`SPARKLE_PRIVATE_KEY`	The 44 characters from step 1
`DEVELOPER_ID_CERT_P12`	The long block of text from step 3
`DEVELOPER_ID_CERT_PASSWORD`	The password you invented in step 2
`NOTARY_APPLE_ID`	Your Apple ID, the email address you sign in with
`NOTARY_PASSWORD`	The app-specific password from step 4

When all five are listed, delete **certificate.p12** from your Desktop and empty the Trash. GitHub holds it now, and a copy sitting on the Desktop is a copy that can be taken.

Ship a release

6 Send the code to GitHub Xcode

1. Open **SuperSubmitter.xcodeproj**.
2. Open the **Source Control** menu and choose **Push…**.
3. In the sheet, set the destination to **github / main**. Not **origin**. That one is GitLab, and it publishes nothing.
4. Choose **Push**.

Xcode will not ask for a password. Your Mac is already signed in.

7 Approve the release github.com

1. Go to the **Actions** tab of the repository. A run named **Release** is waiting.
2. Open it. It says *Review pending deployments*.
3. Choose **Review deployments**, tick **release**, and choose **Approve and deploy**.

About fifteen minutes later the release appears on the Releases page, and every copy of the app already out there will offer it to its owner within a day.

To watch it, stay on the run page. Each step reports as it finishes. If one turns red, open it and read the last few lines: the workflow is written to say what went wrong in plain words.

Every release after the first

Repeat steps 6 and 7. Nothing else.

You can also start a release without sending any new code. On the **Actions** tab, choose **Release** in the sidebar, then the **Run workflow** button. It still waits for your approval.

To change the version number people see, open **project.yml** and edit the `MARKETING_VERSION` line. The build number takes care of itself: it counts the commits, and that count is what tells the app a newer version exists.

What the release does, in order

1. Runs every test. Publishing cannot be undone, so nothing is signed until the tests pass.
2. Builds the app and signs it with your Developer ID certificate.
3. Sends it to Apple to be checked, and staples Apple's approval into the app.
4. Checks four things before it goes out: the build number reached the app, the licensing settings are filled in, the update address matches this repository, and the update list is signed.
5. Writes the release notes from your commit messages since the last release.
6. Signs the update list with your Sparkle key.
7. Publishes the app and the update list together as one GitHub release.

Things that break, and why

Every install stops updating, and nothing says so.

The repository was renamed or deleted. Its address is inside every copy already released and cannot be changed afterwards. There is no repair. Do not rename or delete it.

Every update is refused, and nothing says so.

Somebody made a new Sparkle key. The app only trusts the original. Put the original private key back in the login keychain. Nothing else fixes it.

Apple's check fails.

Usually the app-specific password from step 4 expired or was revoked. Make a new one and update the `NOTARY_PASSWORD` secret. If the message instead names an entitlement, the app asked for a permission it has no profile for, and that is a code change.

The run fails on the signing certificate.

Developer ID certificates expire after five years. Renew it in your Apple Developer account, then redo steps 2, 3 and 5 with the new one.

The run never starts, or never pauses for you.

The `release` environment lost its reviewer. Open **Settings**, then **Environments**, then **release**, and put yourself back under **Required reviewers**.

The build fails and names a macOS version.

GitHub retired the machine the workflow asks for. Open `.github/workflows/release.yml`, find the `runs-on` line, and change it to the current macOS label.